# Solution: Unique Paths

## Problem Statement

Given a 2-dimensional grid of size `m x n` (where `m` is the number of rows and `n` is the number of columns), you need to find out the number of unique paths from the top-left corner to the bottom-right corner. The constraints are that you can only move either right or down at any point in time.

## Examples

1. **Example 1**:
   - **Input**: 3, 3
   - **Expected Output**: 6
   - **Justification**: The six possible paths are:
     1. Right, Right, Down, Down
     2. Right, Down, Right, Down
     3. Right, Down, Down, Right
     4. Down, Right, Right, Down
     5. Down, Right, Down, Right
     6. Down, Down, Right, Right
2. **Example 2**:
   - Input: 3, 2
   - Expected Output: 3
   - Justification: The three possible paths are:
     - Right, right, down
     - Right, down, right
     - Down, right, right
3. **Example 3**:
   - Input: 2, 3
   - Expected Output: 3
   - Justification: The three possible paths are:
     - Down, right, right
     - Right, down, right
     - Right, right, down

## Solution

The unique paths problem can be approached using a dynamic programming solution. Essentially, the idea is to think of the grid as a graph where each cell is a node. Given we can only move right or down, the number of ways to reach a cell is the sum of the number of ways to reach the cell above it and the cell to its left. By breaking down the problem in this way, we can iteratively compute the number of paths to reach any cell, starting from the top-left and working our way to the bottom-right of the grid.

1. **Initialization**:
   - Create a 2-dimensional array `dp` of size `m x n` initialized to zero. This array will store the number of unique paths to reach each cell.
2. **Boundary Cases**:
   - All cells in the first row can only be reached by moving right from the top-left corner. So, the number of unique paths for all cells in the first row will be 1.
   - Similarly, all cells in the first column can only be reached by moving downwards from the top-left corner. So, the number of unique paths for all cells in the first column will be 1.
3. **Filling the Table**:
   - For each remaining cell, the number of unique paths to that cell is the sum of the number of paths from the cell above it and the cell to the left of it. This is because we can only move right or down.
4. **Result**:
   - The bottom-right cell will contain the total number of unique paths from the top-left corner to the bottom-right corner.

## Algorithm Walkthrough

Using the input from Example 1 (2, 2):

- Initialize a 2x2 matrix `dp` with all zeros.

```
0 0
0 0
```

- Fill the first row and first column with 1s.

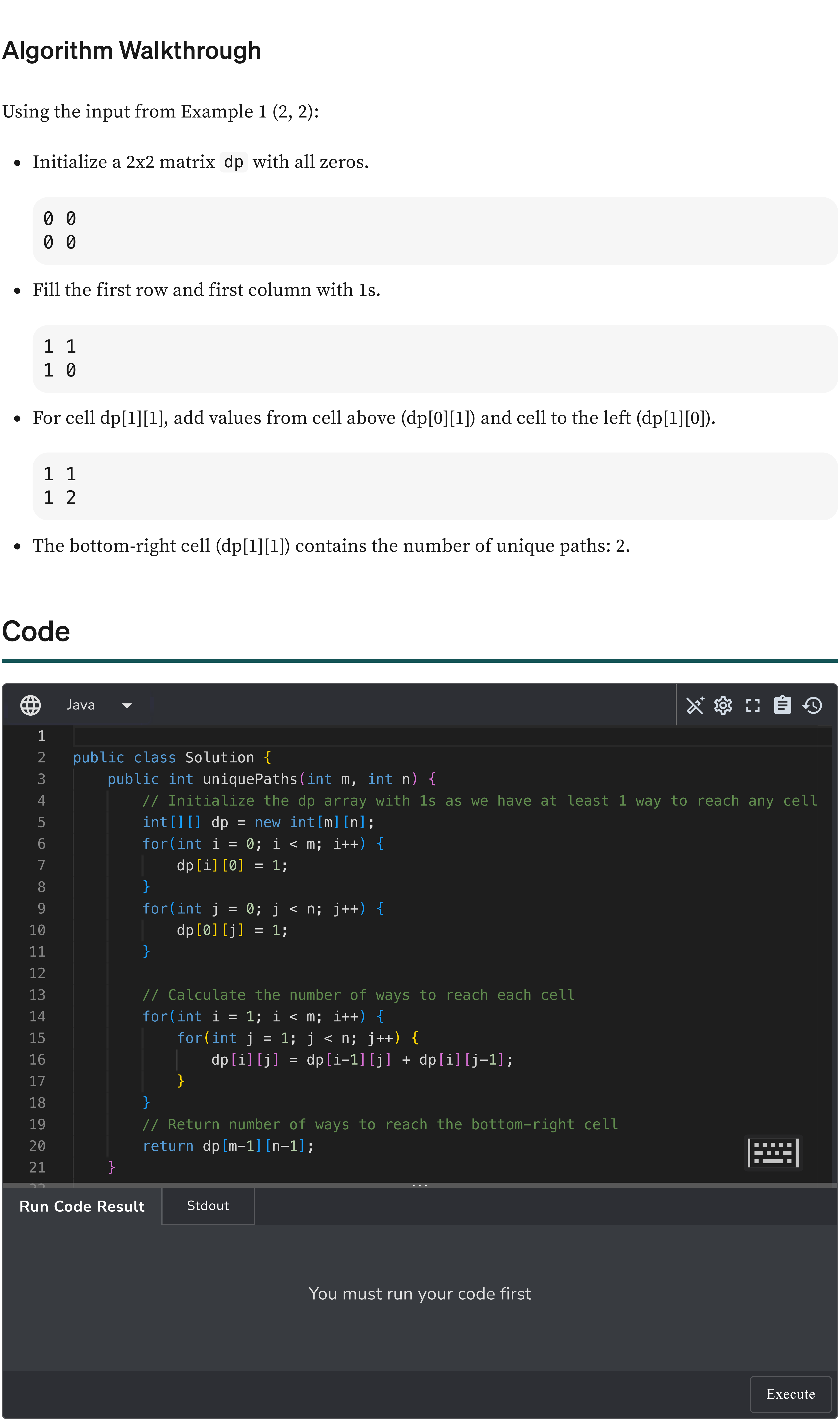
```
1 1
1 0
```

- For cell dp[1][1], add values from cell above (dp[0][1]) and cell to the left (dp[1][0]).

```
1 1
1 2
```

- The bottom-right cell (dp[1][1]) contains the number of unique paths: 2.

## Code



```java

public class Solution {
    public int uniquePaths(int m, int n) {
        // Initialize the dp array with 1s as we have at least 1 way to reach any cell
        int[][] dp = new int[m][n];
        for(int i = 0; i < m; i++) {
            dp[i][0] = 1;
        }
        for(int j = 0; j < n; j++) {
            dp[0][j] = 1;
        }

        // Calculate the number of ways to reach each cell
        for(int i = 1; i < m; i++) {
            for(int j = 1; j < n; j++) {
                dp[i][j] = dp[i-1][j] + dp[i][j-1];
            }
        }
        // Return number of ways to reach the bottom-right cell
        return dp[m-1][n-1];
    }
```

You must run your code first

## Complexity Analysis

- **Time Complexity**: O(m * n) - We are processing each cell once.
- **Space Complexity**: O(m * n) - Due to the 2D dp array.

← Previous
Unique Paths (Medium)

Next →
Word Break (Medium)

☐ Mark as Completed